

16-Hour QA Automation Workshop for BSc CSIT Students

1. Prerequisites & Required Software (Install *Before* Workshop Day)

Send this as a pre-workshop checklist 2–3 days in advance. Reserve the first 10 minutes of Hour 1 for an install-verification check.

Category	Software / Tool	Version / Notes
Programming	Java JDK	JDK 17 or 21 (LTS), java - version to confirm
Build Tool	Apache Maven	3.9+, added to system PATH
IDE	IntelliJ IDEA (Community) or Eclipse	Latest stable
API Testing	Postman (Desktop App)	Latest, free account created in advance
Performance Testing	Apache JMeter	5.6+, requires Java 8+ already installed
Browser	Google Chrome + matching ChromeDriver (or let Selenium Manager handle it)	Selenium 4.6+ auto-manages drivers
Browser (secondary)	Firefox	Optional, for cross-browser demo
Version Control	Git + GitHub account	For sharing capstone project code
Documentation	Google account (Docs/Sheets) or MS Office	For test case documentation exercises
Text/Notes	Any Markdown-capable notes app	Optional
Internet	Stable connection	Required for Postman public APIs, JMeter demo targets

Pre-workshop student tasks:

- Install all software above and verify Java + Maven work via terminal.
 - Create a free Postman account.
 - Create a GitHub account (if not already owned).
 - Bookmark: reqres.in, the-internet.herokuapp.com, demoblaze.com
-

3. Summary Timetable (16 Hours)

Hour	Topic
1	Intro to Testing & QA, SDLC vs STLC
2	Types of Testing
3	Test Case Design Techniques
4	Writing Effective Test Cases
5	Documentation, Bug Reporting & Defect Lifecycle
6	API Testing Fundamentals + Postman Basics
7	Postman — Env Variables, Collections
8	Postman — Collection Runner & Test Scripts
9	Performance Testing Intro + JMeter Setup
10	JMeter — Samplers, Assertions, Timers
11	JMeter — Listeners, Load Test, Reports
12	Selenium Architecture & Environment Setup
13	Selenium — Locators & WebDriver Commands
14	Selenium — Waits, Alerts, Frames, Windows
15	Selenium — Maintainable Scripts & Framework Basics
16	Capstone Mini Project + Presentations + Career Guidance

4. Detailed Hour-by-Hour Breakdown

DAY 1

Hour 1 — Introduction to Software Testing & QA, SDLC vs STLC

Learning Objectives: Understand what testing is, why QA matters, and how testing fits into software development.

Topics Covered: What is testing vs QA vs QC; cost of defects; SDLC phases; STLC phases; how STLC maps onto SDLC.

Theory (≈20 min): Definitions, the "cost of defect" curve (cheaper to fix earlier), SDLC models overview (Waterfall, Agile), STLC phases (requirement analysis, test planning, test case development, environment setup, execution, closure).

Hands-on Practical:

- Students are given a 1-page mock requirement document (e.g., "Login Feature for an E-commerce Site") and, in pairs, identify: what could go wrong, who the stakeholders are, and where testing fits in the timeline.
- Group discussion mapping their answers to SDLC/STLC phases on a shared whiteboard/Google Jamboard.

Tools Used: Whiteboard / Jamboard / Google Docs.

Expected Learning Outcomes: Can explain SDLC vs STLC in their own words; can identify testing touchpoints in a project timeline.

Assignment/Practice: Write a half-page reflection: "Why can't developers alone guarantee software quality?"

Hour 2 — Types of Testing

Learning Objectives: Differentiate functional and non-functional testing types and know when each is used.

Topics Covered: Functional testing, Regression, Smoke, Sanity, Integration, System, UAT, and a brief mention of non-functional testing (performance, security, usability).

Theory (≈15 min): Definitions and a comparison table (Smoke vs Sanity vs Regression is a common interview trap — emphasized here).

Hands-on Practical:

- Case-study worksheet: Given 10 test scenarios (e.g., "verify login after a new build deployment," "verify checkout flow end-to-end," "verify one bug fix didn't break anything"), students classify each into the correct testing type.
- Instructor-led review with live discussion of tricky/ambiguous cases.

Tools Used: Printed/Google Forms worksheet.

Expected Learning Outcomes: Can correctly classify a real-world testing scenario into its testing type — a very common interview question.

Assignment/Practice: Find and screenshot a real bug report/changelog online (e.g., from GitHub issues) and classify what type of testing would have caught it.

Hour 3 — Test Case Design Techniques

Learning Objectives: Apply structured techniques to design test cases with maximum coverage and minimum redundancy.

Topics Covered: Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, State Transition Testing, Error Guessing.

Theory (≈18 min): Explain each technique with a simple example (e.g., age field 18–60 for BVA).

Hands-on Practical:

- Exercise 1: Given an "Age input field (18–60)" — students derive equivalence classes and boundary values.
- Exercise 2: Given a "Discount calculation" business rule (with multiple conditions), students build a decision table.
- Pair review and instructor walkthrough of a model answer.

Tools Used: Spreadsheet (Google Sheets/Excel) for decision tables.

Expected Learning Outcomes: Can independently derive test conditions using at least 2 formal techniques — directly usable in real test case writing.

Assignment/Practice: Apply BVA + Equivalence Partitioning to a "Password field (8–16 characters)" and submit test conditions.

Hour 4 — Writing Effective Test Cases

Learning Objectives: Translate test conditions into clear, executable, professional test cases.

Topics Covered: Test case anatomy (ID, Title, Preconditions, Steps, Test Data, Expected Result, Actual Result, Status, Priority, Severity); characteristics of a good test case (clear, atomic, reusable, traceable).

Theory (≈10 min): Standard test case template walkthrough with a good vs bad example comparison.

Hands-on Practical:

- Students write 8–10 test cases for a login page (using techniques from Hour 3) in a shared Google Sheet test case template.
- Peer review: swap sheets with another student, critique clarity/completeness using a checklist.

Tools Used: Google Sheets / Excel test case template (provided).

Expected Learning Outcomes: Can write a complete, professional test case set for a simple feature without hand-holding.

Assignment/Practice: Write 10 test cases for a "User Registration Form" (with fields: name, email, password, confirm password) and submit before next session.

Hour 5 — Test Case Documentation, Bug Reporting & Defect Lifecycle

Learning Objectives: Document test execution results professionally and write actionable bug reports.

Topics Covered: Test case documentation best practices, traceability matrix basics; bug report anatomy (Title, Steps to Reproduce, Expected vs Actual, Severity vs Priority, Environment, Attachments); Defect Life Cycle (New → Assigned → Open → Fixed → Retest → Closed/Reopened).

Theory (≈15 min): Live example of a poor bug report vs an excellent one; defect life cycle diagram walkthrough.

Hands-on Practical:

- Students are given a deliberately buggy demo web app (e.g., the-internet.herokuapp.com) and must find 3 real bugs and write full bug reports for each.
- Introduce a free bug tracking tool (e.g., **GitHub Issues** or a shared **Jira free-tier board**) — students log at least one bug into the actual tool.

Tools Used: the-internet.herokuapp.com, GitHub Issues (or Jira Free Tier).

Expected Learning Outcomes: Can identify real bugs, write clear reproducible bug reports, and use an issue-tracking tool — a core internship-day-one skill.

Assignment/Practice: Find 2 more bugs on a different demo site (e.g., demoblaze.com) and submit formal bug reports.

Hour 6 — API Testing Fundamentals + Postman Basics

Learning Objectives: Understand REST API fundamentals and perform basic requests in Postman.

Topics Covered: What is an API; HTTP methods (GET, POST, PUT, PATCH, DELETE); anatomy of a request/response; common HTTP status codes (2xx, 3xx, 4xx, 5xx).

Theory (≈15 min): Client-server model diagram; REST principles briefly; status code cheat-sheet.

Hands-on Practical:

- Install/open Postman, create a workspace.
- Send GET, POST, PUT, DELETE requests to the free public API **reqres.in** and observe responses/status codes.
- Students document, for each request: method used, endpoint, status code received, and what it means.

Tools Used: Postman, reqres.in (free public test API).

Expected Learning Outcomes: Can send basic HTTP requests confidently and interpret status codes.

Assignment/Practice: Send all 4 HTTP methods against 2 different endpoints on reqres.in and record results in a table.

Hour 7 — Postman: Headers, Request Body, Environment Variables, Collections

Learning Objectives: Work with headers/body data and organize requests using variables and collections.

Topics Covered: Request headers (Content-Type, Authorization); JSON request bodies; Environment & global variables; creating and organizing Collections; folder structure best practices.

Theory (≈8 min): Why environment variables matter (dev/staging/prod switching) — quick real-world analogy.

Hands-on Practical:

- Create a Postman Environment with variables (base_url, token).
- Build a Collection for a sample API (reqres.in or a public weather API) with 5–6 organized requests using {{base_url}}.
- Add a custom header and a JSON body to a POST request.

Tools Used: Postman.

Expected Learning Outcomes: Can structure a real testing project in Postman the way professionals do — reusable, environment-driven collections.

Assignment/Practice: Build a full Collection (min. 6 requests) for a public API of their choice with at least one environment configured.

Hour 8 — Postman: Collection Runner & Basic Test Scripts

Learning Objectives: Automate execution of multiple requests and write basic assertions/test scripts.

Topics Covered: Postman test scripts (JavaScript-based pm.test), common assertions (status code, response time, JSON field value), Collection Runner for batch execution.

Theory (≈8 min): Where test scripts run in the request lifecycle (Pre-request vs Tests tab).

Hands-on Practical:

- Add test scripts to 3–4 requests: assert status code = 200, assert response time < 500ms, assert a JSON field equals expected value.
- Run the entire Collection using Collection Runner and review the pass/fail summary report.
- Export the Collection + Environment as JSON (deliverable for later use).

Tools Used: Postman (Tests tab, Collection Runner).

Expected Learning Outcomes: Can write basic automated API test assertions and execute/interpret a batch test run — a genuine entry-level API automation skill.

Assignment/Practice (End of Day 1 Homework): Add test scripts to their full Collection from Hour 7's homework; run via Collection Runner; screenshot the results summary.

DAY 2

Hour 9 — Performance Testing Fundamentals + JMeter Setup, Test Plans, Thread Groups

Learning Objectives: Understand performance testing concepts and build a basic JMeter Test Plan.

Topics Covered: Load vs Stress vs Spike vs Endurance testing; key metrics (response time, throughput, error rate); JMeter GUI overview; Test Plan structure; Thread Groups (users, ramp-up, loop count).

Theory (≈15 min): Why performance testing matters (real-world outage examples); JMeter architecture at a glance.

Hands-on Practical:

- Open JMeter, create a new Test Plan.
- Add a Thread Group configured for 10 users, 10-second ramp-up, 1 loop.
- Save the Test Plan (.jmx file).

Tools Used: Apache JMeter.

Expected Learning Outcomes: Comfortable navigating the JMeter GUI and understands Thread Group parameters.

Assignment/Practice: Reflect: "What would happen to a website if 10,000 users hit it in 1 second?" (2–3 sentences).

Hour 10 — JMeter: HTTP Request Samplers, Assertions, Timers

Learning Objectives: Configure real HTTP requests in JMeter and add validation logic.

Topics Covered: HTTP Request Sampler configuration (server, path, method); Response Assertions; Constant/Random Timers to simulate real user think-time.

Theory (≈8 min): How samplers + assertions + timers combine into a realistic simulated user.

Hands-on Practical:

- Add an HTTP Request Sampler targeting a public test API (e.g., reqres.in or httpbin.org).
- Add a Response Assertion checking for status code 200 and a text pattern in the response.
- Add a Constant Timer (e.g., 1000ms delay) between requests.

Tools Used: Apache JMeter, reqres.in / httpbin.org.

Expected Learning Outcomes: Can build a functioning, validated HTTP test step in JMeter.

Assignment/Practice: Add 2 more HTTP samplers (different endpoints) with assertions to the same Test Plan.

Hour 11 — JMeter: Listeners, Executing Load Tests, Reading Reports

Learning Objectives: Execute a load test and interpret the results meaningfully.

Topics Covered: Listeners (View Results Tree, Summary Report, Aggregate Report); running tests in GUI vs recommended non-GUI (CLI) mode; interpreting key metrics (average/min/max response time, throughput, error %).

Theory (≈8 min): Why GUI mode is only for test design, not real load generation (CLI mode is best practice).

Hands-on Practical:

- Add View Results Tree + Aggregate Report listeners.

- Run the Test Plan with 20 users, ramp-up 10s, and observe results.
- Increase to 50 users and compare — students identify degradation, if any, in response time/error rate.
- (If time allows) Run the same plan via CLI (`jmeter -n -t test.jmx -l results.jtl`) and generate an HTML dashboard report.

Tools Used: Apache JMeter (GUI + CLI).

Expected Learning Outcomes: Can execute a load test and produce/read a performance report — able to explain "what does this report tell us" in plain language.

Assignment/Practice: Run their own Test Plan at 3 different load levels (10, 30, 60 users) and write a 5-sentence summary comparing results.

Hour 12 — Selenium Architecture & Environment Setup

Learning Objectives: Understand Selenium's architecture and set up a working Java + Maven + Selenium project.

Topics Covered: Selenium WebDriver architecture (client libraries, browser drivers, W3C WebDriver protocol); why Selenium 4 auto-manages drivers; Maven project structure (pom.xml, dependencies).

Theory (≈20 min): Architecture diagram (test script → language binding → driver → browser); Maven basics for those unfamiliar.

Hands-on Practical:

- Create a new Maven project in IntelliJ/Eclipse.
- Add Selenium + JUnit/TestNG dependencies to pom.xml.
- Write and run a "Hello Selenium" script: open Chrome, navigate to a URL, print the page title, close browser.
- Troubleshoot common setup issues live (PATH issues, driver version mismatches) as a class.

Tools Used: IntelliJ IDEA/Eclipse, Maven, Selenium WebDriver, Chrome.

Expected Learning Outcomes: Has a fully working Selenium project scaffold — the single biggest blocker for beginners removed early.

Assignment/Practice: Modify the "Hello Selenium" script to open 3 different websites in sequence and print each title.

Hour 13 — Selenium: Locators & WebDriver Commands

Learning Objectives: Locate web elements reliably and perform core WebDriver interactions.

Topics Covered: Locator strategies (ID, Name, ClassName, CSS Selector, XPath, LinkText); WebDriver commands (get, click, sendKeys, getText, isDisplayed, navigation commands).

Theory (≈8 min): Locator priority best practice (ID > CSS > XPath as last resort); using browser DevTools to inspect elements.

Hands-on Practical:

- Using the-internet.herokuapp.com/login: locate username/password fields via different strategies, fill the form, click submit, and assert the success message text.
- Automate a second flow: the-internet.herokuapp.com/dropdown navigation menu interactions.

Tools Used: Selenium WebDriver, Chrome DevTools, the-internet.herokuapp.com.

Expected Learning Outcomes: Can locate elements confidently using multiple strategies and automate a simple end-to-end form flow.

Assignment/Practice: Automate the login flow with both a valid and an invalid credential scenario, asserting the correct message in each case.

Hour 14 — Selenium: Waits, Alerts, Frames, Windows, and Dropdowns

Learning Objectives: Handle dynamic and complex UI elements reliably.

Topics Covered: Implicit vs Explicit vs Fluent Waits; handling JavaScript Alerts/Confirms/Prompts; switching between iframes; handling multiple windows/tabs; Select class for dropdowns.

Theory (≈8 min): Why hard-coded Thread.sleep() is a beginner anti-pattern; how Explicit Wait (WebDriverWait + ExpectedConditions) solves flakiness.

Hands-on Practical:

- Automate a JS Alert scenario (accept/dismiss/read text) on the-internet.herokuapp.com/javascript_alerts.
- Automate an iframe interaction (/iframe demo page).
- Automate a multiple-windows scenario (/windows demo page), switching handles.
- Automate a dropdown selection using the Select class.
- Replace any Thread.sleep() in earlier scripts with proper Explicit Waits.

Tools Used: Selenium WebDriver, the-internet.herokuapp.com.

Expected Learning Outcomes: Can handle the most common "tricky" UI automation scenarios that trip up beginners in real internships.

Assignment/Practice: Automate a full scenario combining a dropdown selection + alert handling + window switch in one script.

Hour 15 — Selenium: Writing Maintainable Scripts & Basic Framework Structure

Learning Objectives: Refactor automation scripts for maintainability and understand basic framework design.

Topics Covered: Page Object Model (POM) concept; separating test logic from locators; reusable utility methods; brief intro to TestNG/JUnit annotations (@Test, @BeforeMethod, @AfterMethod); industry best practices (naming conventions, avoiding hard waits, meaningful assertions, version control for test code).

Theory (≈8 min): Why raw scripts don't scale in real projects; POM diagram (Page classes vs Test classes).

Hands-on Practical:

- Refactor the login automation script from Hour 13 into a simple Page Object class (LoginPage.java) + a separate Test class (LoginTest.java).
- Push the completed Maven project to a GitHub repository.
- Group discussion: industry best-practice checklist review (naming, comments, folder structure, .gitignore for target/ folder).

Tools Used: IntelliJ/Eclipse, Maven, TestNG/JUnit, Git & GitHub.

Expected Learning Outcomes: Understands *why* frameworks exist and can structure code the way real QA teams do — huge differentiator in internship interviews.

Assignment/Practice: Convert 2 more scripts from earlier hours into the Page Object structure before the capstone session.

Hour 16 — Capstone Mini Project, Presentations & Career Guidance

Learning Objectives: Integrate all workshop skills into one deliverable and understand the path to a QA Automation career.

Topics Covered: End-to-end mini project execution; presentation of results; QA career paths; resume/portfolio tips; interview preparation.

Structure of this hour (accelerated capstone — see Section 5 for full capstone spec, ideally started earlier and wrapped up here):

- **First 25 min:** Final work time — teams finish test case docs, bug reports, Postman collection, JMeter results, and Selenium scripts for the assigned demo site (e.g., demoblaze.com).
- **Next 25 min:** Each team (3–5 min each) presents: requirements analyzed, test cases written, bugs found, API test results, performance test summary, and a live/recorded Selenium run.
- **Last 10 min:** Career guidance — QA career ladder (Manual QA → SDET → Automation Engineer → QA Lead), resume tips, must-have GitHub portfolio pieces, common interview questions, and how to keep practicing after the workshop.

Tools Used: All workshop tools + presentation slides/GitHub repo links.

Expected Learning Outcomes: Has one complete, presentable QA project spanning test design, API testing, performance testing, and UI automation — ready to reference in interviews and internship applications.

Assignment/Practice (Post-Workshop): Publish the capstone project to GitHub with a proper README describing scope, tools, and how to run the tests; add it to LinkedIn/resume as a project.

5. Capstone Mini Project — Full Specification

Suggested target app: demoblaze.com (free, stable, demo e-commerce site) —
alternative: any instructor-approved demo app.

Team size: 2–3 students.

Deliverables:

1. **Requirement Analysis (0.5 hr, can start end of Day 1 as homework seed):** Pick one user flow (e.g., "search product → add to cart → checkout") and write a short 1-paragraph feature description as if it were a real requirement doc.
2. **Test Case Documentation:** Minimum 15 test cases covering the chosen flow, using the techniques and template from Hours 3–4, stored in a shared Google Sheet.
3. **Bug Reporting:** Minimum 3 real bugs found on the demo site, logged as GitHub Issues with full repro steps.
4. **API Testing:** A Postman Collection (min. 5 requests) against a public API related or unrelated to the UI flow (e.g., reqres.in), with at least 3 test scripts/assertions, executed via Collection Runner with a passing report.
5. **Performance Testing:** A JMeter Test Plan simulating 20–50 concurrent users against a chosen endpoint, with an Aggregate Report screenshot and a 3–4 sentence interpretation.
6. **UI Automation:** A Selenium (Java + Maven) script automating the full chosen user flow end-to-end, using at least one wait strategy, one assertion, and (if possible) structured using a basic Page Object.
7. **Presentation:** A short 3–5 minute walkthrough (slides optional) covering all of the above, with the GitHub repo link shared.

Grading/Completion Rubric (optional for instructor use):

Criteria	Weight
Test case quality & documentation	20%
Bug reports (clarity, reproducibility)	15%
API testing (coverage, assertions)	20%
Performance testing (setup, interpretation)	15%
Selenium automation (functionality, code quality)	20%

Criteria	Weight
Presentation clarity	10%

6. Recommended Free Websites & Tools for Practice (During & After Workshop)

Practice targets for manual + automated testing:

- the-internet.herokuapp.com — best beginner UI automation playground (alerts, frames, dropdowns, dynamic loading, file upload, etc.)
- demoblaze.com — demo e-commerce app for full flow automation
- saucedemo.com — Sauce Labs' demo login/e-commerce app, popular in tutorials/interviews
- automationexercise.com — has both a UI and a documented public API, great for combined practice
- [opencart.com demo store] — real-world-like OpenCart demo installs, used in many QA courses

Free public APIs for Postman/JMeter practice:

- reqres.in — fake but structured REST API with users/auth endpoints
- jsonplaceholder.typicode.com — fake REST API for posts/comments/users
- httpbin.org — great for exploring headers, status codes, methods
- [restful-booker](https://restful-booker.herokuapp.com/) — booking API with auth, ideal for advanced Postman practice

Bug tracking / documentation practice:

- GitHub Issues (free, familiar to students already)
- Jira Free Tier (up to 10 users) — closer to real industry tool
- Google Sheets test case templates (free, no setup)

Learning/reference resources:

- Postman Learning Center (official docs)
- Apache JMeter official user manual
- Selenium official documentation (selenium.dev)

- Ministry of Testing (free articles/community, great for QA career content)
-

7. Homework / Practice Exercises After Each Major Session (Consolidated)

After Session	Exercise
SDLC/STLC (Hr 1)	Reflection: role of QA in preventing costly defects
Types of Testing (Hr 2)	Classify a real-world bug/changelog by testing type
Test Design Techniques (Hr 3)	Apply BVA + EP to a password field
Writing Test Cases (Hr 4)	Write 10 test cases for a registration form
Bug Reporting (Hr 5)	Find & report 2 bugs on a new demo site
Postman Basics (Hr 6)	Send all 4 HTTP methods to 2 endpoints, record results
Postman Collections (Hr 7)	Build a 6-request Collection with an environment
Postman Test Scripts (Hr 8)	Add assertions to Collection, run via Collection Runner
JMeter Setup (Hr 9)	Reflect on real-world load scenarios
JMeter Samplers (Hr 10)	Add 2 more validated HTTP samplers
JMeter Reports (Hr 11)	Compare 3 load levels, write summary
Selenium Setup (Hr 12)	Extend Hello Selenium script to 3 sites
Selenium Locators (Hr 13)	Automate valid + invalid login scenarios
Selenium Waits/Alerts (Hr 14)	Combine dropdown + alert + window switch in one script
Selenium POM (Hr 15)	Refactor 2 more scripts into Page Object structure
Capstone (Hr 16)	Publish full project to GitHub with README

8. Common Beginner Mistakes & How to Avoid Them

Mistake	Why It Happens	How to Avoid
Writing vague test cases ("test login works")	Skipping test design techniques	Always derive test conditions from EP/BVA/decision tables first
Confusing Severity with Priority in bug reports	Not taught the distinction early	Emphasize: Severity = impact on system, Priority = urgency to fix
Automating before understanding manual testing	Excitement to "code" over "think"	Enforce the manual-first sequence in this workshop; don't skip Hours 1–5
Using <code>Thread.sleep()</code> everywhere in Selenium	Easiest fix that "seems to work"	Teach Explicit Waits early and ban <code>Thread.sleep()</code> in code reviews
Fragile XPath locators (absolute paths)	Copy-pasting from browser DevTools	Teach relative CSS/XPath and ID-first locator strategy
Running JMeter load tests in GUI mode for real load	GUI is the only mode they've seen	Explicitly teach and demo CLI mode for actual load generation
Not resetting Postman environment variables between runs	Not understanding variable scope	Explain Environment vs Global vs Collection variable scope clearly
No version control for automation code	Treating scripts as throwaway	Require Git commits from Hour 12 onward, push capstone to GitHub
Writing one giant test method with no structure	No exposure to frameworks	Teach Page Object Model, even if basic, by Hour 15
Ignoring flaky test failures	Assuming "it'll pass next time"	Teach that flakiness is a signal, not noise — investigate root cause

9. Tips for Internship & Entry-Level QA Interview Preparation

Technical preparation:

- Be ready to explain the difference between Smoke, Sanity, and Regression testing — asked in almost every QA interview.

- Practice writing test cases live on a whiteboard/Google Doc for a simple feature (e.g., "test a calculator's addition button").
- Be able to explain the Defect Life Cycle end-to-end, including who does what at each stage.
- Know basic SQL (SELECT, JOIN, WHERE) — many QA roles expect basic database verification skills even if not covered deeply in this workshop.
- Be comfortable explaining what a REST API is, and the meaning of common status codes (200, 201, 400, 401, 404, 500) without looking them up.
- Practice explaining a Selenium script you wrote, line by line, out loud — interviewers often ask "walk me through your code."

Portfolio & resume:

- Publish the capstone project (and any homework scripts) to GitHub with clear READMEs — this is the single biggest differentiator for freshers with no work experience.
- List specific tools with proficiency honestly (e.g., "Postman: comfortable with collections, environments, basic test scripts" rather than just "Postman").
- Include metrics where possible: "Wrote 25+ test cases," "Automated 5 end-to-end flows," "Found and reported 8 bugs in a demo application."

Soft skills & mindset:

- QA is about communication as much as technical skill — practice explaining a bug clearly to a non-technical person.
- Show curiosity: mention side practice on sites like the-internet.herokuapp.com or personal bug bounty-style exploration of demo apps.
- Be honest about being a fresher — emphasize eagerness to learn frameworks/tools used at the specific company, since most companies expect on-the-job ramp-up.
- Ask good questions in interviews: "What testing tools/frameworks does your QA team currently use?" shows genuine interest.

After the workshop — keep practicing:

- Recreate this workshop's exercises on a new demo site every week for a month.

- Contribute to or fork open-source Selenium/Postman example frameworks on GitHub to read real-world code.
 - Follow QA communities (Ministry of Testing, r/QualityAssurance, LinkedIn QA groups) for job postings and trends.
-

10. Facilitator Notes (Optional, for Whoever Delivers This Workshop)

- Keep a shared "parking lot" doc for setup issues (driver mismatches, JDK path issues) so troubleshooting doesn't eat into teaching time — assign a TA if possible for Hour 12.
- Pre-test all demo websites (the-internet.herokuapp.com, demoblaze.com, reqres.in) on workshop morning — these free community-hosted sites occasionally have downtime.
- Provide all templates (test case sheet, bug report template, Postman collection starter, Selenium Maven starter project) as ready-to-clone resources so students spend time testing, not scaffolding.
- Consider recording Hour 12 (environment setup) separately and sharing beforehand, since setup issues are the #1 time-sink in any hands-on Selenium session.